

УТВЕРЖДЕН

RU.17701729.06.05-01 81 01-1-ЛУ

ИНТЕЛЛЕКТУАЛЬНЫЙ ПЛАНИРОВЩИК ЗАДАЧ

Пояснительная записка

RU.17701729.06.05-01 81 01-1

Листов 32

Инв.№ подп	Подп. и дата	Взам. инв.№	Инв.№ дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	3
1.1. Наименование программы	3
1.2. Тема разработки	3
1.3. Документ, на основании которого ведется разработка	3
2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	4
2.1. Назначение программы	4
2.2. Краткая характеристика области применения программы	4
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	5
3.1. Постановка задачи на разработку программы	5
3.2. Описание архитектуры программного решения	5
3.3. Описание алгоритма функционирования программы	10
3.4. Описание метода организации входных и выходных данных	19
3.5. Описание состава технических и программных средств	20
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	22
4.1. Ориентировочная экономическая эффективность	22
4.2. Предполагаемая потребность	22
4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными образцами или аналогами	22
ПРИЛОЖЕНИЕ 1. ССЫЛКИ НА АНАЛОГИ	23
ПРИЛОЖЕНИЕ 2. ФУНКЦИОНАЛЬНЫЙ АНАЛИЗ КОНКУРЕНТОВ	24
ПРИЛОЖЕНИЕ 3. АРХИТЕКТУРА ПРОГРАММНОГО ПРОЕКТА	25
ПРИЛОЖЕНИЕ 4. СТРУКТУРА ТАБЛИЦ И СВЯЗИ БАЗЫ ДАННЫХ	26
ПРИЛОЖЕНИЕ 5. ПЕРЕЧЕНЬ ТЕРМИНОВ	27
ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ	30

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Интеллектуальный планировщик задач».

Наименование программы на английском языке – «Intelligent Task Planner».

Краткое наименование программы – «ToDoAI».

1.2. Тема разработки

Интеллектуальный планировщик задач — веб-приложение, предназначенное для создания, хранения и сопровождения пользовательских задач, учета текущего состояния пользователя, автоматической генерации дневного расписания и анализа фактического выполнения задач с использованием алгоритмов интеллектуального планирования.

1.3. Документ, на основании которого ведется разработка

Основанием для разработки является учебный план подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденная академическим руководителем программы тема курсового проекта.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Назначение программы

Назначение программы заключается в повышении эффективности личного планирования и организации повседневной деятельности пользователя. Приложение автоматизирует процессы постановки задач, формирования дневного плана, контроля выполнения и учета затраченного времени. Веб-интерфейс предоставляет удобные средства для взаимодействия с задачами, а использование интеллектуального модуля позволяет адаптировать расписание с учетом приоритетов, сроков и текущего состояния пользователя.

2.2. Краткая характеристика области применения программы

Программа будет применима в области персональной продуктивности и цифровых систем поддержки планирования. Она может использоваться для организации учебной, рабочей и личной деятельности, когда требуется формирование оптимального плана выполнения задач с учетом ограниченности времени, уровня нагрузки и индивидуальных параметров состояния пользователя.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Целью разработки является создание интеллектуального веб-приложения для управления задачами и планирования дня с использованием технологий искусственного интеллекта.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать backend-сервис для управления пользователями, задачами, пользовательским состоянием, расписанием и результатами выполнения задач;
2. Реализовать frontend-приложение для взаимодействия пользователя с системой через web-интерфейс;
3. Обеспечить возможность регистрации, авторизации, подтверждения электронной почты и восстановления доступа пользователя;
4. Реализовать CRUD-операции для задач, включая создание, редактирование, удаление и получение списка задач;
5. Реализовать механизм отслеживания выполнения задач, включая фиксацию статусов, учет затраченного времени и сохранение истории рабочих сессий;
6. Реализовать механизм работы с пользовательским состоянием, включающим показатели энергии, стресса, мотивации, концентрации и продолжительности сна;
7. Реализовать механизм формирования дневного расписания задач на основе текущего состояния пользователя, приоритетов задач и доступного времени;
8. Интегрировать AI-сервис для интеллектуального ранжирования задач при построении расписания и генерации мотивационных сообщений;
9. Обеспечить взаимодействие backend-сервиса и AI-сервиса через HTTP API;
10. Реализовать хранение пользовательских данных, задач, расписаний, рабочих сессий и истории выполнения в базе данных PostgreSQL;
11. Обеспечить контейнеризацию backend, frontend, AI-сервиса и базы данных с использованием Docker;
12. Реализовать API для взаимодействия frontend-приложения с backend-сервисом;
13. Организовать конфигурацию и запуск всех модулей проекта в единой среде развёртывания.

3.2. Описание архитектуры программного решения

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

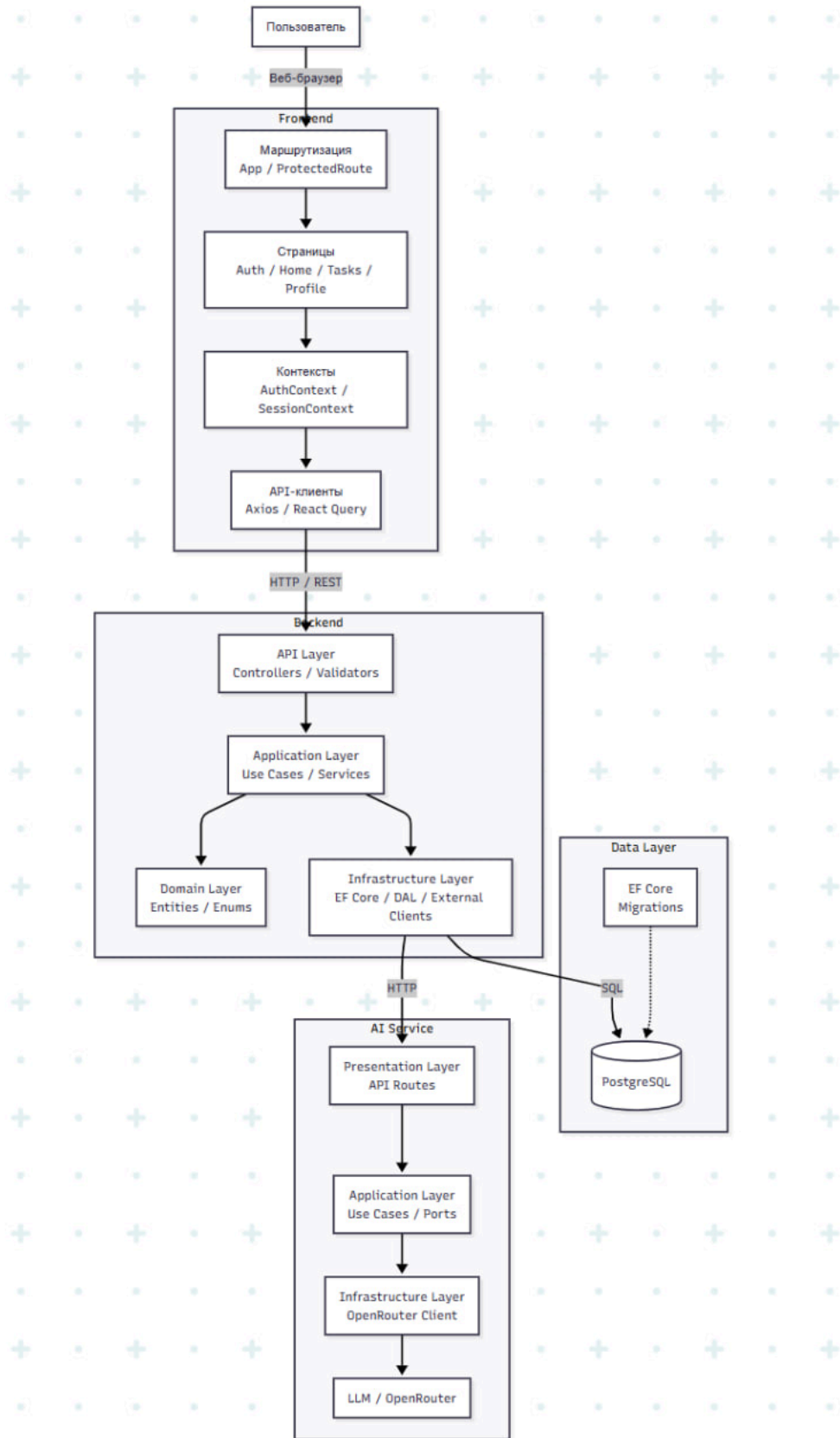


Рис. 1. Архитектура проекта

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Архитектура данного проекта реализована в виде многокомпонентной клиент-серверной системы, включающей frontend-приложение, backend-сервис, отдельный AI-сервис и базу данных PostgreSQL. Такое построение позволяет разделить функции пользовательского интерфейса, бизнес-логики, интеллектуальной обработки и долговременного хранения данных между независимыми компонентами системы.

Frontend-часть реализована как web-приложение и предназначена для непосредственного взаимодействия пользователя с системой. Через пользовательский интерфейс выполняются регистрация и вход в систему, создание и редактирование задач, ввод текущего состояния пользователя, запуск генерации дневного расписания, просмотр сформированного плана, а также фиксация фактического выполнения задач.

Backend-сервис является центральным компонентом системы и реализует основную прикладную логику. Именно он принимает HTTP-запросы от frontend-приложения, выполняет аутентификацию и авторизацию пользователей, обрабатывает операции над задачами, обеспечивает работу с пользовательским состоянием, организует хранение и получение расписаний, а также координирует взаимодействие с AI-сервисом. Backend также отвечает за валидацию входных данных, применение бизнес-правил и сохранение результатов в базе данных.

Серверная часть проекта построена по многослойной архитектуре. В backend-сервисе можно выделить API-слой, отвечающий за прием HTTP-запросов, маршрутизацию и преобразование внешних моделей запросов и ответов, слой бизнес-логики, реализующий ключевые сценарии работы системы, инфраструктурный слой, обеспечивающий доступ к базе данных, применение миграций и интеграцию с внешними сервисами, а также доменный слой, содержащий основные сущности предметной области и перечисления, описывающие состояние объектов системы. Подобное разделение уменьшает связанность компонентов, упрощает тестирование и позволяет независимо изменять отдельные части системы.

Для обеспечения устойчивости внешнего интерфейса backend-сервис использует версионирование API, благодаря чему маршруты приложения имеют фиксированный формат обращения вида `api/v1.0/`. Для основных групп входных запросов, включая аутентификацию, операции над задачами, работу с пользовательским состоянием и генерацию расписания, используются отдельные валидаторы, проверяющие корректность передаваемых параметров до выполнения бизнес-операций.

В составе предметной области выделяются несколько основных сущностей: пользователь, задача, пользовательское состояние, дневное расписание, блок расписания, рабочая сессия и факт

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

выполнения задачи. Задача содержит описание работы, приоритет, оценку длительности, сложность, срок выполнения и текущий статус. Пользовательское состояние фиксирует показатели, которые влияют на планирование дня, например уровень энергии, стресса, мотивации, концентрации и продолжительность сна. Дневное расписание состоит из набора временных блоков, каждый из которых соответствует определенной задаче.

AI-сервис выделен в отдельный модуль, предназначенный для интеллектуального ранжирования задач и формирования рекомендаций при построении плана дня. Выделение этого компонента в отдельный сервис позволяет изолировать логику, связанную с обращением к языковой модели, от основной серверной части. Это упрощает сопровождение проекта и позволяет при необходимости изменять используемую модель или сам алгоритм интеллектуального планирования без существенного изменения backend-сервиса.

Внутри AI-сервиса также реализовано разделение на слои: слой представления принимает HTTP-запросы, прикладной слой реализует сценарий генерации расписания, доменный слой содержит модели планирования, а инфраструктурный слой отвечает за обращение к внешнему провайдеру языковой модели.

При построении расписания backend-сервис выступает координатором процесса: он подготавливает данные, необходимые для планирования, передает их в AI-сервис и сохраняет полученный результат в базе данных. При этом интеллектуальный модуль остается независимым компонентом общей архитектуры системы.

Между компонентами системы реализованы следующие способы взаимодействия:

1. Между frontend-приложением и backend-сервисом обмен осуществляется через REST API;
2. Между backend-сервисом и базой данных взаимодействие осуществляется через PostgreSQL и Entity Framework Core;
3. Между backend-сервисом и AI-сервисом обмен осуществляется через HTTP запросы в формате JSON;
4. Между AI-сервисом и внешним провайдером языковой модели выполняются HTTP-запросы к OpenRouter API.

Для хранения данных используется реляционная база PostgreSQL. В ней сохраняются учетные записи пользователей, задачи, история пользовательских состояний, версии дневных расписаний, временные блоки расписания, рабочие сессии и записи о фактическом выполнении задач. Такая

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

структура позволяет не только поддерживать текущее состояние системы, но и использовать накопленную историю при последующем построении расписания.

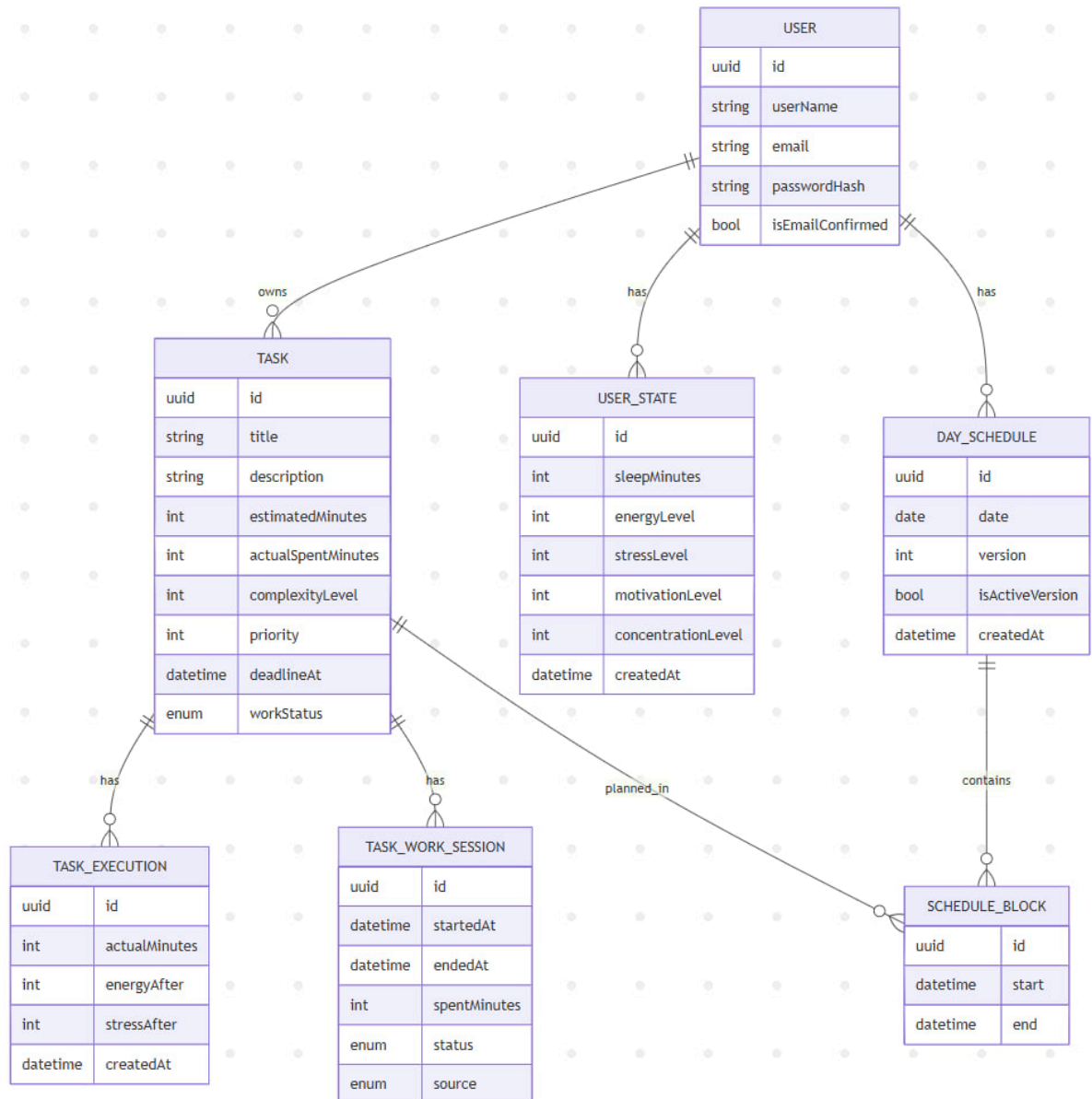


Рис. 2. Упрощенная модель данных

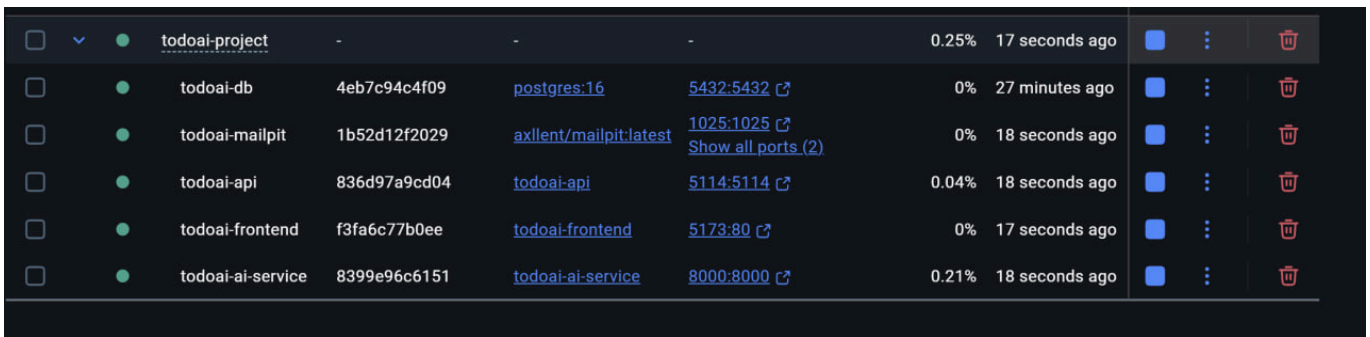
Пользователь может иметь множество задач, множество записей о своем состоянии и множество версий дневного расписания. Каждая задача связана с конкретным пользователем и может участвовать в одном или нескольких расписаниях. Кроме того, задача может иметь связанные рабо-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

чие сессии и записи о фактическом завершении, содержащие информацию о реально затраченном времени и изменении состояния пользователя после выполнения.

Для дневных расписаний поддерживается механизм версионирования: каждая новая генерация плана на одну и ту же дату может сохраняться как отдельная версия, при этом в системе выделяется активная версия расписания, используемая как основная для отображения и дальнейшей работы пользователя.

Контейнеризация проекта реализована с использованием Docker. Каждый основной компонент системы запускается в изолированном контейнере, что упрощает развертывание, тестирование и перенос проекта между средами. Использование контейнеров также позволяет независимо запускать frontend, backend, AI-сервис и базу данных, сохраняя единый способ сборки и запуска всего проекта.



Container Name	ID	Image	Ports	Usage	Status	Time	Actions
todoai-project	-	-	-	0.25%	Running	17 seconds ago	Stop, Restart, Delete
todoai-db	4eb7c94c4f09	postgres:16	5432:5432	0%	Running	27 minutes ago	Stop, Restart, Delete
todoai-mailpit	1b52d12f2029	axllent/mailpit:latest	1025:1025	0%	Running	18 seconds ago	Stop, Restart, Delete
todoai-api	836d97a9cd04	todoai-api	5114:5114	0.04%	Running	18 seconds ago	Stop, Restart, Delete
todoai-frontend	f3fa6c77b0ee	todoai-frontend	5173:80	0%	Running	17 seconds ago	Stop, Restart, Delete
todoai-ai-service	8399e96c6151	todoai-ai-service	8000:8000	0.21%	Running	18 seconds ago	Stop, Restart, Delete

Рис. 3. Состав контейнеров проекта в Docker

Подобная архитектура обеспечивает модульность, расширяемость, удобство сопровождения и устойчивость системы к сбоям отдельных компонентов. Она хорошо подходит для интеллектуального приложения, в котором необходимо сочетать стандартную серверную обработку, хранение пользовательских данных и отдельный механизм работы с моделями искусственного интеллекта.

3.3. Описание алгоритма функционирования программы

Функционирование программы начинается с развертывания всех основных компонентов проекта командой:

```
docker compose up --build
```

На данном этапе выполняются сборка и запуск контейнеров frontend-приложения, backend-сервиса, AI-сервиса, базы данных PostgreSQL, а также вспомогательных компонентов среды разработки. Использование Docker позволяет запускать систему в воспроизводимом окружении и упрощает настройку зависимостей между отдельными сервисами.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

=> [api] exporting to image 0.0s
=> exporting layers 0.0s
=> writing image sha256:279bf29ba437ee91438303c0ad55134af4cbcefb0bf9d939eb607eba10fc43d9 0.0s
=> naming to docker.io/library/todoai-api 0.0s
=> [api] resolving provenance for metadata file 0.0s
[+] Running 4/4
  ✓ todoai-api Built 0.0s
  ✓ todoai-frontend Built 0.0s
  ✓ todoai-ai-service Built 0.0s
  ✓ Container todoai-db Running 0.0s
Attaching to todoai-ai-service, todoai-api, todoai-db, todoai-frontend, todoai-mailpit
todoai-mailpit | time="2026/04/26 22:08:56" level=info msg="[smtpd] starting on [::]:1025 (no encryption)"
todoai-mailpit | time="2026/04/26 22:08:56" level=info msg="[cors] allowed API origins: "
todoai-mailpit | time="2026/04/26 22:08:56" level=info msg="[http] starting on [::]:8025"
todoai-mailpit | time="2026/04/26 22:08:56" level=info msg="[http] accessible via http://localhost:8025/"
todoai-frontend | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
todoai-frontend | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
todoai-frontend | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
todoai-frontend | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
todoai-frontend | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged versi
on
todoai-frontend | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
todoai-frontend | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
todoai-frontend | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
todoai-frontend | /docker-entrypoint.sh: Configuration complete; ready for start up
todoai-frontend | 2026/04/26 22:08:57 [notice] 1#1: using the "epoll" event method
todoai-frontend | 2026/04/26 22:08:57 [notice] 1#1: nginx/1.27.5
todoai-frontend | 2026/04/26 22:08:57 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
todoai-frontend | 2026/04/26 22:08:57 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
todoai-frontend | 2026/04/26 22:08:57 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576

```

Рис. 4. Логи сборки и запуска контейнеров проекта

После запуска backend-сервис выполняет инициализацию приложения, настройку маршрутов API, конфигурацию служб, подключение к базе данных и применение миграций. После этого система становится готовой к обработке пользовательских запросов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

Containers / todoai-api

todoai-api
836d97a9cd04 todoai-api:latest
5114:5114

STATUS
Running (29 seconds ago)

Logs Inspect Bind mounts Exec Files Stats

"MigrationId" character varying(150) NOT NULL,
"ProductVersion" character varying(32) NOT NULL,
CONSTRAINT "PK__EFMigrationsHistory" PRIMARY KEY ("MigrationId")
);
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  LOCK TABLE "ToDoAIService"."__EFMigrationsHistory" IN ACCESS EXCLUSIVE MODE
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  SELECT "MigrationId", "ProductVersion"
  FROM "ToDoAIService"."__EFMigrationsHistory"
  ORDER BY "MigrationId";
info: Microsoft.EntityFrameworkCore.Migrations[20405]
  No migrations were applied. The database is already up to date.
App is starting...
warn: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[60]
  Storing keys in a directory '/root/.aspnet/DataProtection-Keys' that may not be persisted outside of the container. Protected data will be
  unavailable when container is destroyed. For more information go to https://aka.ms/aspnet/dataprotectionwarning
warn: Microsoft.AspNetCore.Hosting.Diagnostics[15]
  Overriding HTTP_PORTS '8080' and HTTPS_PORTS ''. Binding to values defined by URLS instead 'http://+:5114'.
info: Microsoft.Hosting.Lifetime[14]
  Now listening on: http://[::]:5114
info: Microsoft.Hosting.Lifetime[0]
  Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
  Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
  Content root path: /app
  
```

Рис. 5. Логи запуска backend-сервиса и применения миграций

Работа пользователя с системой начинается с регистрации, подтверждения электронной почты и последующей аутентификации. После успешного входа пользователь получает доступ к функциональности планирования и управления задачами.

Безопасность пользовательского взаимодействия обеспечивается за счет применения схемы аутентификации с access- и refresh-токенами, хранением токенов сессии, а также механизма CSRF-защиты для серверных операций, изменяющих состояние данных. Такой подход уменьшает риск несанкционированного доступа и повышает защищенность пользовательской сессии.

Важным системным сценарием является регистрация пользователя и подтверждение адреса электронной почты. На этапе регистрации frontend-приложение передает введенные пользователем данные в backend-сервис. Серверная часть проверяет уникальность имени пользователя и адреса электронной почты, создает учетную запись, сохраняет данные для подтверждения и инициирует отправку письма с кодом подтверждения. После ввода пользователем кода backend выполняет его проверку и, в случае успешного совпадения, помечает адрес электронной почты как подтвержденный.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

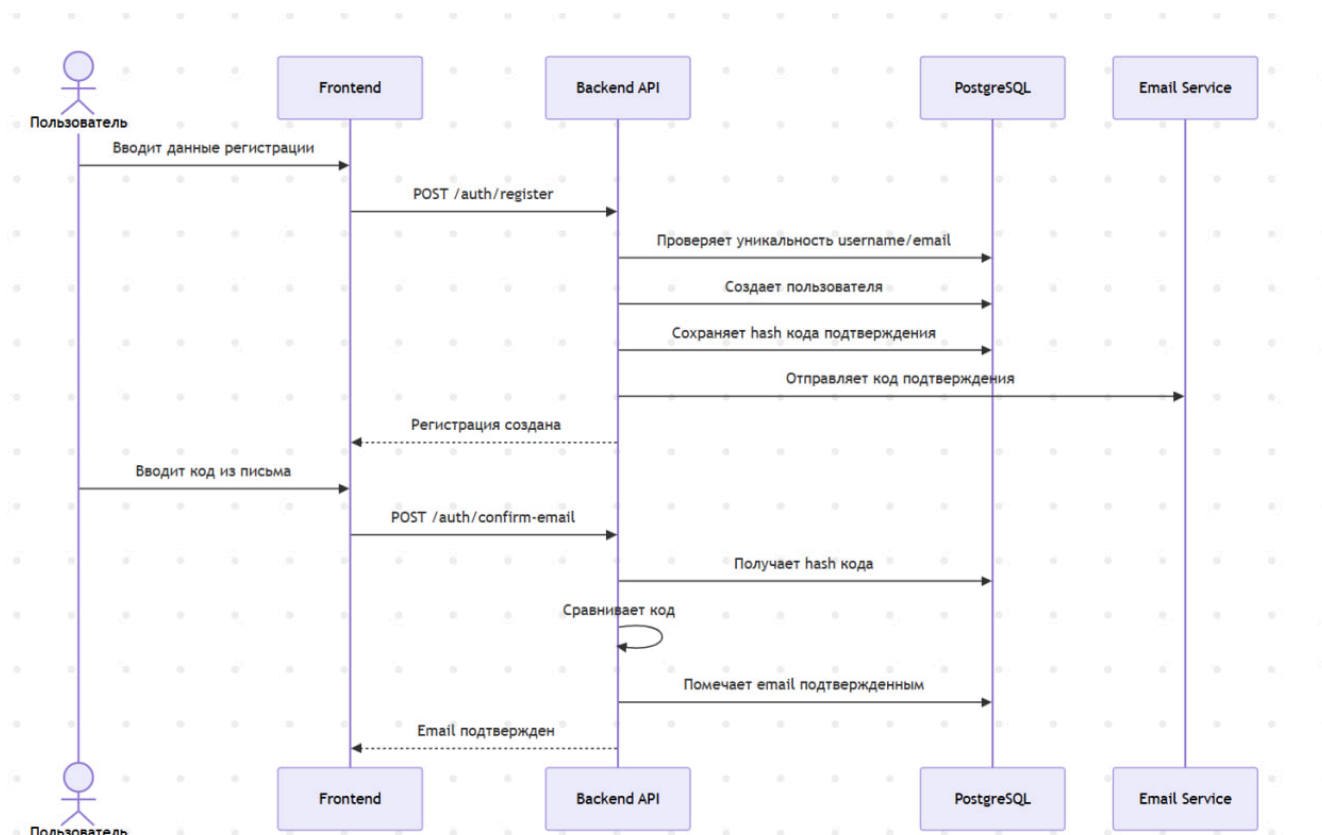


Рис. 6. Сценарий регистрации и подтверждения электронной почты

Дополнительным важным сценарием является восстановление доступа к учетной записи. Пользователь вводит адрес электронной почты, после чего backend формирует и сохраняет данные для восстановления пароля и инициирует отправку письма с кодом. После ввода кода и нового пароля backend проверяет корректность и срок действия кода, обновляет пароль пользователя и удаляет использованные данные восстановления.

В среде разработки отправка служебных сообщений электронной почты выполняется через вспомогательный почтовый сервис Mailpit. Это позволяет тестировать сценарии регистрации, подтверждения адреса электронной почты и восстановления пароля без использования внешнего почтового провайдера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

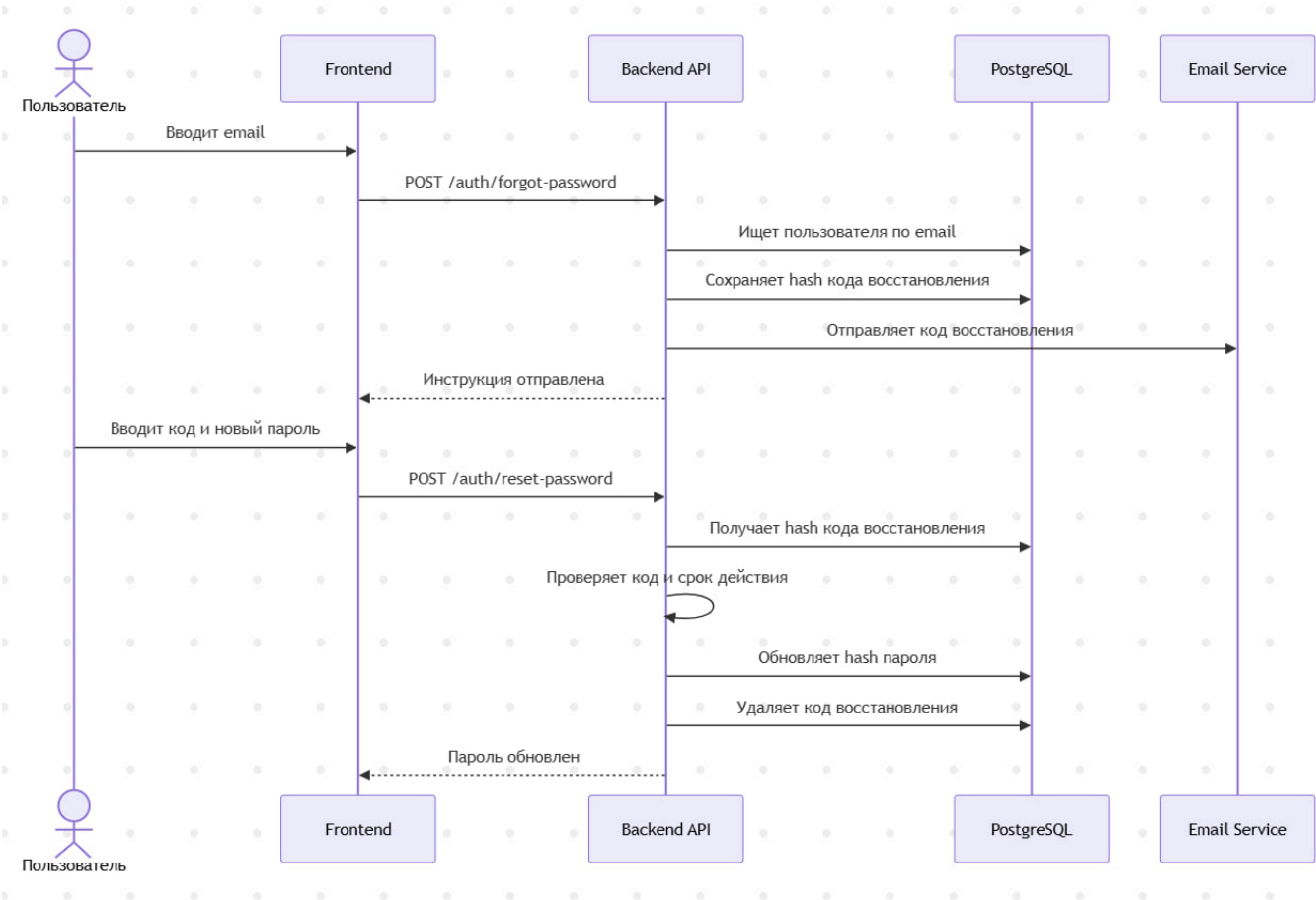


Рис. 7. Сценарий восстановления пароля

Следующим этапом является формирование пользовательского набора задач. Пользователь может создавать задачи, указывая их название, описание, приоритет, сложность, предполагаемую длительность и срок выполнения. Каждая задача получает определенный статус жизненного цикла, который используется системой при отображении и дальнейшем планировании.

Перед построением расписания пользователь должен зафиксировать свое текущее состояние. Для этого в систему передаются значения, характеризующие уровень энергии, стресса, мотивации, концентрации и количество сна. Эти показатели сохраняются в базе данных и затем используются как один из главных факторов при определении рекомендуемого порядка выполнения задач.

Формирование дневного расписания начинается после того, как пользователь выбирает задачи и задает дату планирования. Frontend-приложение отправляет запрос в backend-сервис. Получив этот запрос, backend выполняет несколько последовательных действий:

- 1. Проверяет корректность входных данных и права доступа пользователя;
- 2. Извлекает из базы данных выбранные задачи;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- 3. Получает последнюю запись о состоянии пользователя;
- 4. Извлекает недавнюю историю выполнения задач;
- 5. Проверяет отсутствие активной рабочей сессии, которая могла бы конфликтовать с построением нового плана;
- 6. Формирует запрос в AI-сервис.

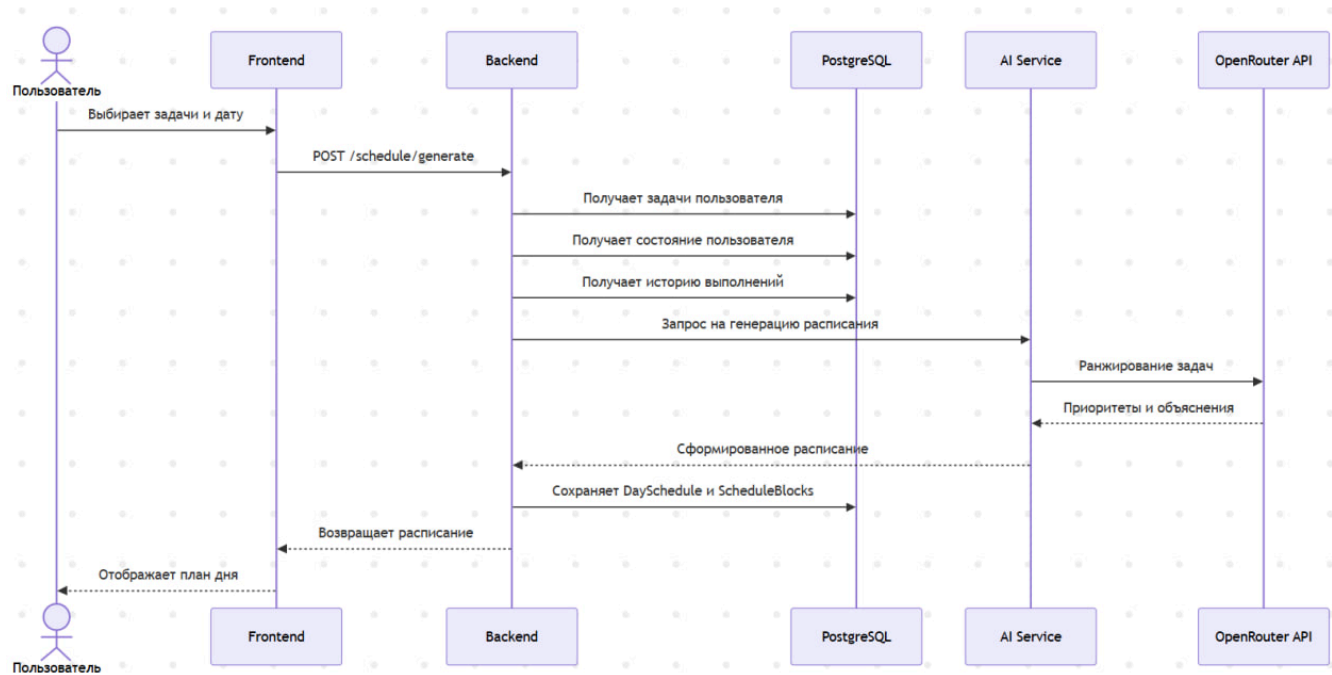


Рис. 8. Флоу генерации расписания

AI-сервис принимает подготовленный backend-запрос и преобразует его во внутреннее представление. Далее формируется обращение к внешней языковой модели через OpenRouter API. В запрос передаются характеристики задач, их оставшееся время, приоритет, сложность, сроки, а также текущее состояние пользователя и сведения о последних выполненных задачах. На основе этих данных модель определяет рекомендуемый порядок задач и формирует краткие объяснения, почему выбран именно такой вариант распределения.

После получения ответа AI-сервиса backend анализирует его и строит расписание на день в виде набора временных блоков. Если внешний интеллектуальный компонент оказался недоступен либо вернул некорректный результат, backend использует резервный алгоритм. В резервном режиме задачи сортируются по приоритету, дедлайнам и доступному времени, что позволяет системе продолжить работу без полной остановки функции планирования.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

После формирования итогового плана backend сохраняет его в базе данных как новую версию дневного расписания и набор временных блоков. Это позволяет хранить историю сформированных планов и использовать ее для последующего анализа. После сохранения готовый результат возвращается во frontend-приложение, где отображается пользователю в виде плана дня.

Дополнительно в системе реализован механизм сопровождения выполнения задач. Пользователь может начать рабочую сессию по задаче, остановить ее или отменить. Такие действия фиксируются в базе данных и позволяют отслеживать реальную длительность выполнения работы. После завершения задачи пользователь может указать фактически затраченное время, а также собственное состояние после выполнения, например уровень энергии и стресса. Эти сведения используются как обратная связь для следующих запусков интеллектуального планирования.

Ключевым пользовательским сценарием является создание задачи, запуск ее выполнения и фиксация результата. Пользователь создает задачу через frontend-приложение, после чего backend выполняет валидацию параметров и сохраняет новую запись в базе данных. При начале работы по задаче создается рабочая сессия, а статус задачи изменяется на состояние выполнения. После завершения пользователь отправляет данные о фактическом результате, backend сохраняет сведения о выполнении и при необходимости запрашивает у AI-сервиса мотивационное сообщение.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

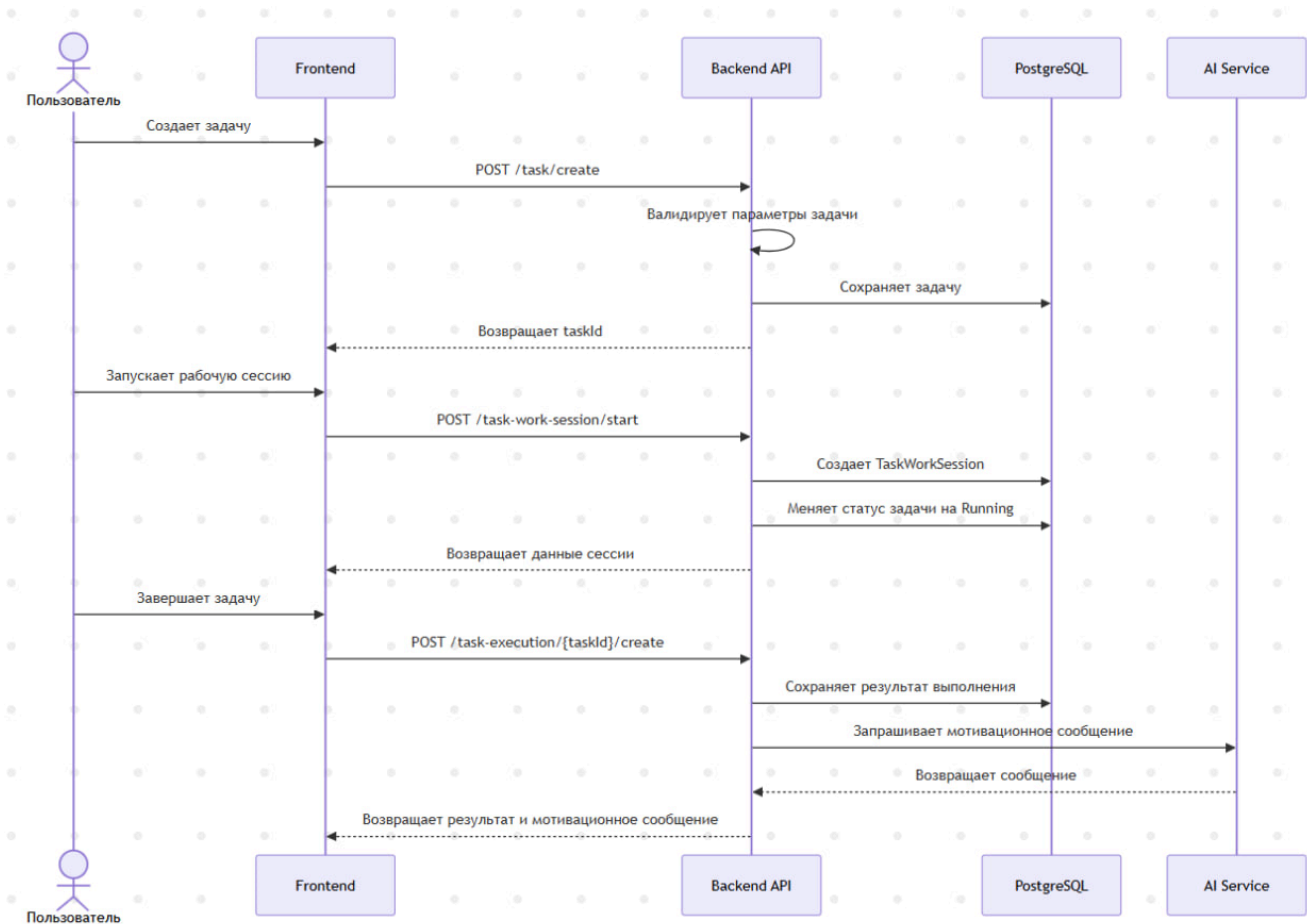


Рис. 9. Сценарий создания и выполнения задачи

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

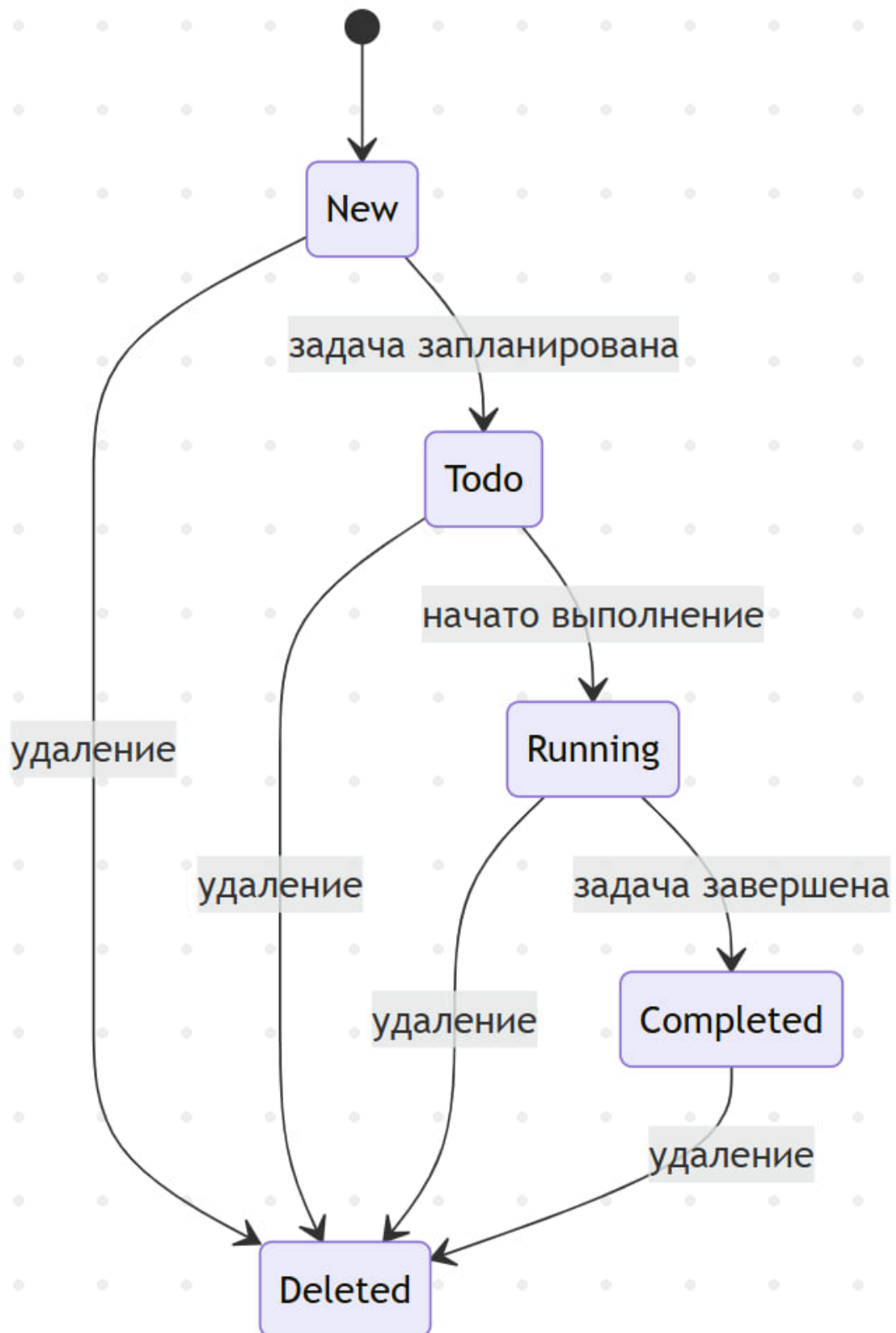


Рис. 10. Жизненный цикл задачи

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Жизненный цикл задачи в системе включает несколько основных состояний: новая задача, запланированная задача, задача в процессе выполнения, завершенная задача и удаленная задача. После создания задача находится в начальном состоянии. При включении в дневное расписание она переходит в состояние запланированной. В момент начала фактической работы задача получает статус выполняемой. После завершения она переводится в состояние завершенной. На любом из промежуточных этапов задача может быть удалена пользователем, если она больше не актуальна.

Параллельно с этим осуществляется отображение актуальной информации в пользовательском интерфейсе. Пользователь может просматривать список задач, текущий план дня, историю рабочих сессий и иные связанные данные.

Алгоритм функционирования программы объединяет ввод пользовательских данных, централизованную обработку бизнес-логики, хранение результатов в базе данных и использование отдельного интеллектуального модуля для адаптивного планирования дня. Это позволяет системе не только фиксировать задачи пользователя, но и активно помогать в их упорядочивании и выполнении.

3.4. Описание метода организации входных и выходных данных

Входными данными для программы являются данные, вводимые пользователем через web-интерфейс, а также параметры, передаваемые между внутренними сервисами системы. К пользовательским входным данным относятся регистрационные и аутентификационные данные, сведения о задачах, значения пользовательского состояния, параметры генерации расписания, данные о рабочих сессиях и результаты фактического выполнения задач.

На уровне взаимодействия frontend-приложения с backend-сервисом входные данные передаются в виде HTTP-запросов. В качестве форматов входных структур используются JSON-объекты, содержащие параметры операций создания учетной записи, подтверждения электронной почты, входа в систему, восстановления пароля, создания и изменения задач, генерации дневного расписания, запуска и завершения рабочих сессий, а также фиксации результатов выполнения задач.

На уровне взаимодействия backend-сервиса с AI-сервисом входными данными являются структурированные JSON-модели, включающие перечень задач, сведения о приоритетах, сроках выполнения и оставшемся времени, текущие показатели состояния пользователя и информацию о недавней истории выполнения задач. Эти данные используются интеллектуальным модулем для ранжирования задач и формирования рекомендаций при построении плана дня.

При развертывании системы дополнительными входными данными выступают конфигурационные параметры и переменные окружения. К ним относятся адреса сервисов, параметры подклю-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

чения к базе данных, настройки JWT-аутентификации, конфигурация почтового сервиса, параметры интеграции с AI-сервисом и ключи доступа к внешнему провайдеру языковой модели.

Выходными данными программы являются результаты обработки запросов, возвращаемые пользователю и внутренним сервисам системы. Основным форматом выходных данных является JSON. В этом формате frontend-приложение получает сведения о пользователе, списке задач, состоянии задач, дневном расписании, рабочих сессиях, результатах выполнения задач и сообщениях интеллектуального модуля.

Часть выходных данных представляется пользователю в преобразованном виде через графический интерфейс. Например, данные о расписании отображаются в виде плана дня с временными блоками, данные о задачах — в виде карточек и списков, а сведения о выполнении — в виде истории рабочих сессий и результатов завершения задач. Кроме того, отдельным видом выходных данных являются служебные сообщения электронной почты, используемые для подтверждения адреса пользователя и восстановления доступа к учетной записи.

3.5. Описание состава технических и программных средств

Клиентская часть:

Для минимальной работы веб-приложения на стороне пользователя требуется следующий состав технических средств:

1. Персональный компьютер, ноутбук, планшет или иное устройство с установленной операционной системой: Windows 10 и выше, macOS 11 и выше, Linux, Android или iOS;
2. Современный веб-браузер с поддержкой JavaScript, cookie-файлов и защищенного сетевого взаимодействия;
3. Не менее 2 ГБ оперативной памяти для комфортной работы браузера и пользовательского интерфейса;
4. Наличие стабильного подключения к сети Интернет для взаимодействия с серверной частью приложения.

Серверная часть:

Для минимальной работы серверной части программного комплекса требуется следующий состав технических средств:

1. Сервер или персональный компьютер с установленной операционной системой: Linux, Windows 10/11, macOS или иная ОС, поддерживающая Docker;
2. Процессор с поддержкой одновременного запуска нескольких контейнеров;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. Не менее 8 ГБ оперативной памяти для параллельной работы frontend, backend, AI-сервиса и базы данных PostgreSQL;
4. Не менее 10 ГБ свободного места на диске для размещения исходного кода, контейнерных образов, зависимостей и данных базы;
5. Сетевое подключение с доступом к внешним сервисам, необходимым для работы почтового сервиса и интеграции с OpenRouter API.

Для развертывания и сопровождения программного комплекса используются следующие программные средства:

1. Docker и Docker Compose для сборки и запуска контейнеров проекта;
2. PostgreSQL 16 в качестве системы управления базами данных;
3. .NET SDK 10 для сборки и запуска backend-сервиса;
4. Python 3.11 и выше для работы AI-сервиса;
5. Poetry для управления зависимостями Python-модуля;
6. Node.js и npm для сборки frontend-приложения;
7. Git для управления исходным кодом проекта.

В состав программных средств проекта также входят используемые фреймворки и библиотеки:

1. React, TypeScript и Vite для реализации клиентской части;
2. ASP.NET Core и Entity Framework Core для реализации backend-сервиса;
3. FastAPI, Uvicorn и httpx для реализации AI-сервиса;
4. OpenRouter API как внешний провайдер языковой модели;
5. Mailpit как вспомогательный почтовый сервис в среде разработки.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

В рамках данного проекта расчет экономической эффективности не предусмотрен.

4.2. Предполагаемая потребность

Интеллектуальный планировщик задач может быть востребован пользователями, которым необходимо системно организовывать учебную, рабочую и личную деятельность. Программа может применяться студентами, преподавателями, офисными сотрудниками, фрилансерами и другими категориями пользователей, заинтересованными в цифровой поддержке планирования, контроле выполнения задач и адаптации дневного расписания к текущему состоянию и уровню нагрузки.

4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными образцами или аналогами

Разрабатываемое приложение представляет собой программное решение, объединяющее классические функции управления задачами с интеллектуальным механизмом формирования дневного расписания, учетом состояния пользователя и анализом фактического выполнения задач. Использование веб-интерфейса и контейнеризированной архитектуры упрощает развертывание и сопровождение системы, а модульная структура позволяет развивать отдельные компоненты независимо друг от друга. Сравнительный анализ функциональных возможностей программы по сравнению с аналогами представлен в ПРИЛОЖЕНИИ 2.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1. ССЫЛКИ НА АНАЛОГИ

Приложение	Ссылка
Todoist	https://todoist.com/
Microsoft To Do	https://todo.microsoft.com/
Notion	https://www.notion.so/
TickTick	https://ticktick.com/
Any.do	https://www.any.do/
Google Tasks	https://tasks.google.com/

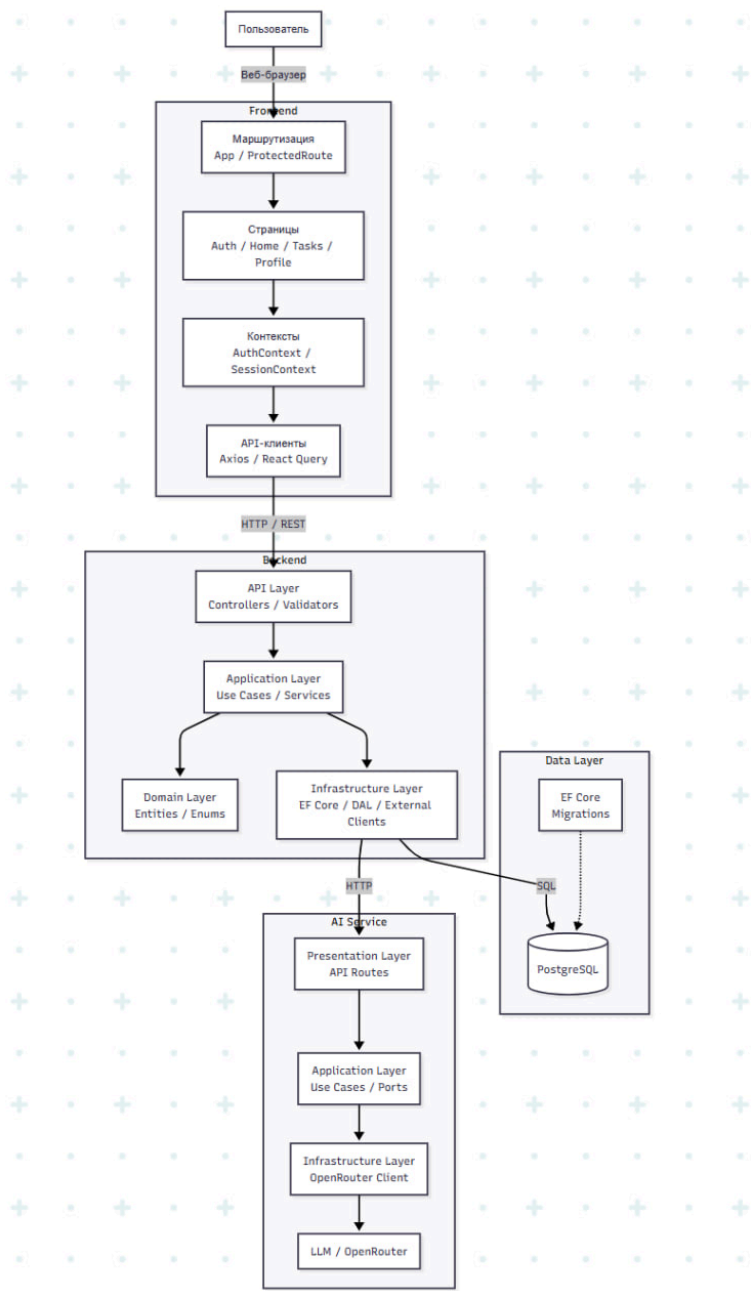
Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2. ФУНКЦИОНАЛЬНЫЙ АНАЛИЗ КОНКУРЕНТОВ

Функция	Todoist	Microsoft To Do	Notion	TickTick	Any.do	Google Tasks	ToDoAI
Работа через веб-браузер	+	+	+	+	+	+	+
Авторизация пользователя	+	+	+	+	+	+	+
Хранение данных в учетной записи	+	+	+	+	+	+	+
Просмотр списка задач	+	+	+	+	+	+	+
Просмотр подробной информации о задаче	+	+	+	+	+	+	+
Изменение статуса или отметка выполнения задачи	+	+	+	+	+	+	+
Создание и управление задачами	+	+	+	+	+	+	+
Дедлайны задач	+	+	+	+	+	+	+
Уведомления и напоминания	+	+	+	+	+	+	+
Автоматическая генерация и перегенерация дневного расписания	-	-	-	-	-	-	+
Сбор обратной связи о выполнении задач	-	-	-	-	-	-	+
Учет состояния пользователя	-	-	-	-	-	-	+
Учет фактического выполнения задач	-	-	-	-	-	-	+
Мотивирующие сообщения	-	-	-	-	-	-	+
Отслеживание рабочих сессий	-	-	-	+	+	-	+
Итого	9	9	9	10	10	9	15

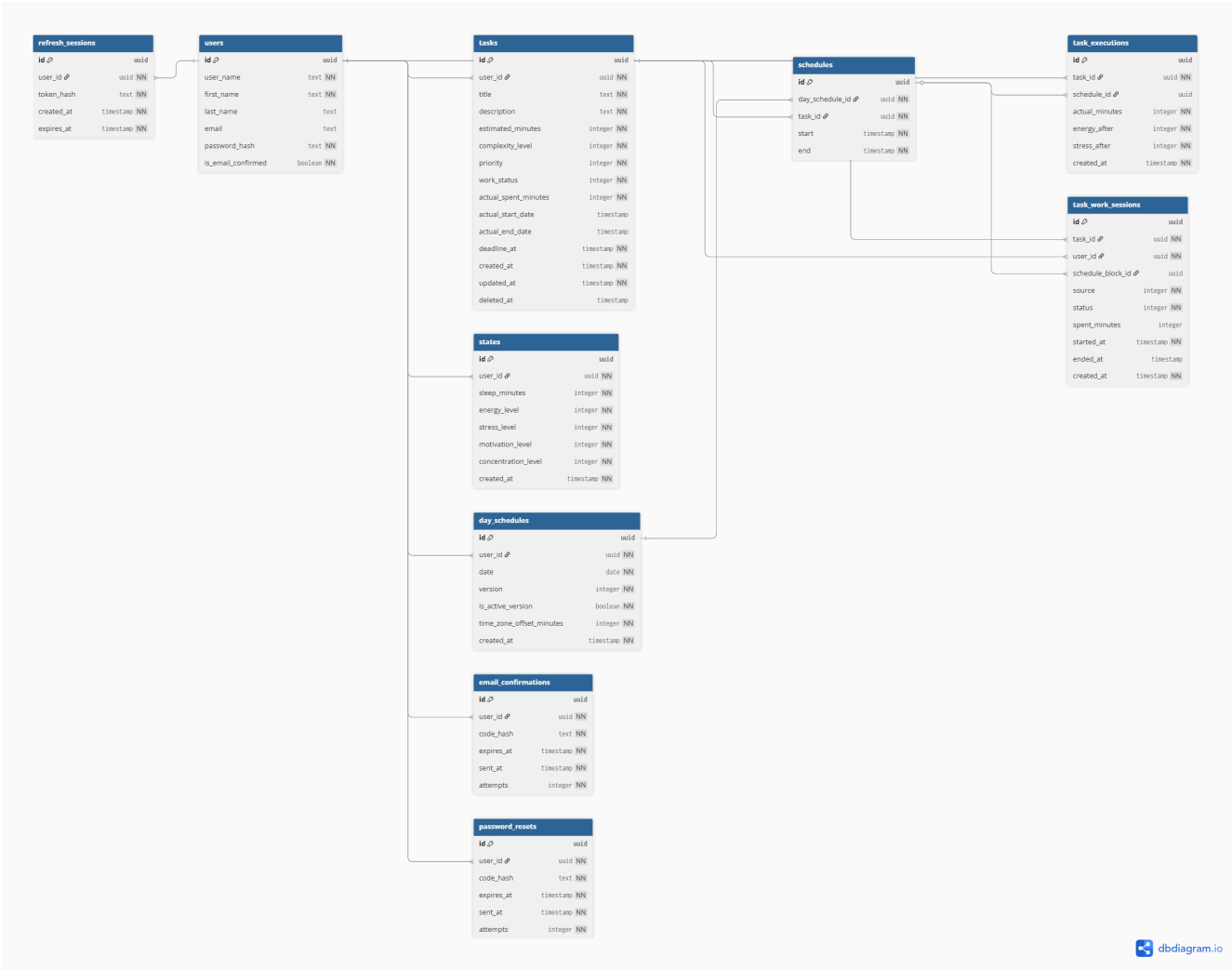
Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3. АРХИТЕКТУРА ПРОГРАММНОГО ПРОЕКТА



Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4. СТРУКТУРА ТАБЛИЦ И СВЯЗИ БАЗЫ ДАННЫХ



UML-диаграмма таблиц базы данных и их связей

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 5. ПЕРЕЧЕНЬ ТЕРМИНОВ

1. Задача — единица пользовательской деятельности, подлежащая планированию и выполнению, характеризуемая названием, описанием, предполагаемой длительностью, приоритетом, уровнем сложности, сроком выполнения и текущим статусом.
2. Дневное расписание — упорядоченный по времени план выполнения задач пользователя на выбранную дату.
3. Блок расписания — элемент дневного расписания, содержащий временной интервал, выделенный на выполнение конкретной задачи.
4. Состояние пользователя — совокупность параметров, характеризующих текущее состояние пользователя, включая продолжительность сна, уровень энергии, стресса, мотивации и концентрации.
5. Рабочая сессия — интервал времени, в течение которого пользователь фактически выполняет задачу.
6. Результат выполнения задачи — сведения, вводимые пользователем после завершения задачи, включая фактическое время выполнения и показатели состояния после выполнения.
7. Пользовательская сессия — состояние авторизованного взаимодействия пользователя с программой после успешного входа в систему.
8. Аутентификация — процедура проверки подлинности пользователя по предоставленным учетным данным.
9. Авторизация — процесс предоставления пользователю доступа к функциям и данным системы после успешной аутентификации.
10. Refresh-токен — специальный токен, используемый для продления пользовательской сессии без повторного ввода учетных данных.
11. Подтверждение электронной почты — процедура проверки принадлежности адреса электронной почты пользователю с использованием кода подтверждения.
12. Восстановление пароля — процедура обновления учетных данных пользователя с использованием временного кода, отправленного на электронную почту.
13. API (Application Programming Interface) — программный интерфейс, обеспечивающий взаимодействие между компонентами системы, в том числе между клиентской и серверной частями приложения, а также между backend-сервисом и AI-сервисом.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

14. Frontend — клиентская часть web-приложения, отвечающая за отображение пользовательского интерфейса и взаимодействие с пользователем.
15. Backend — серверная часть приложения, реализующая бизнес-логику, обработку запросов, доступ к данным и интеграцию с внешними сервисами.
16. AI-сервис — отдельный программный модуль, предназначенный для интеллектуального ранжирования задач, поддержки генерации расписания и формирования текстовых рекомендаций.
17. React — библиотека JavaScript/TypeScript, используемая для разработки пользовательского интерфейса клиентской части приложения.
18. TypeScript — язык программирования, используемый для реализации клиентской части приложения.
19. C# — язык программирования, используемый для реализации серверной части приложения на платформе .NET.
20. .NET — программная платформа, используемая для разработки и выполнения серверной части приложения.
21. Python — язык программирования, используемый для реализации AI-сервиса и логики интеллектуального анализа.
22. FastAPI — фреймворк на языке Python, используемый для реализации HTTP API AI-сервиса.
23. PostgreSQL — реляционная система управления базами данных, используемая для хранения пользовательских данных, задач, расписаний, рабочих сессий и истории выполнения.
24. Entity Framework Core — средство объектно-реляционного отображения, используемое в серверной части приложения для работы с базой данных.
25. Миграция базы данных — описание изменения структуры базы данных, применяемое для создания и обновления схемы хранения данных.
26. Docker — платформа контейнеризации, используемая для сборки, развертывания и запуска компонентов программного комплекса.
27. Docker Compose — средство оркестрации контейнеров, используемое для совместного запуска frontend-приложения, backend-сервиса, AI-сервиса, базы данных и вспомогательных компонентов.
28. LLM (Large Language Model) — большая языковая модель, используемая для интеллектуальной обработки данных, ранжирования задач и генерации текстовых результатов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

29. OpenRouter — внешний сервис доступа к языковым моделям, используемый AI-сервисом приложения.
30. Валидатор — программный компонент, выполняющий проверку корректности входных данных перед выполнением бизнес-операции.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. React [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 27.02.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

15. TypeScript [Электронный ресурс]. URL: <https://www.typescriptlang.org/> (дата обращения: 27.02.2026).
16. Vite [Электронный ресурс]. URL: <https://vite.dev/> (дата обращения: 27.02.2026).
17. .NET [Электронный ресурс]. URL: <https://dotnet.microsoft.com/> (дата обращения: 27.02.2026).
18. ASP.NET Core [Электронный ресурс]. URL: <https://learn.microsoft.com/aspnet/core/> (дата обращения: 27.02.2026).
19. Entity Framework Core [Электронный ресурс]. URL: <https://learn.microsoft.com/ef/core/> (дата обращения: 27.02.2026).
20. Python [Электронный ресурс]. URL: <https://www.python.org/> (дата обращения: 27.02.2026).
21. FastAPI [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 27.02.2026).
22. PostgreSQL [Электронный ресурс]. URL: <https://www.postgresql.org/> (дата обращения: 27.02.2026).
23. Docker [Электронный ресурс]. URL: <https://www.docker.com/> (дата обращения: 27.02.2026).
24. OpenRouter [Электронный ресурс]. URL: <https://openrouter.ai/> (дата обращения: 27.02.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]